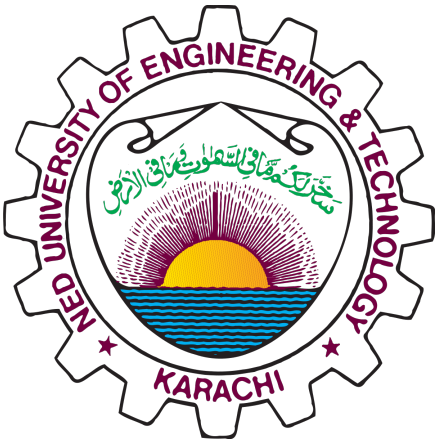


NED University
of Engineering and Technology

Team Reference Document

Team $(ttl)^3$

DP

0-1 Knapsack You are given n items with i_{th} item having weight w_i and value v_i. Given a knapsack with max Weight capacity W. Find the max value you can have.

Note that each item can only be taken **once**.

Time Complexity: $O(nW)$ **Space Complexity:** $O(W)$

```
int knapsack(int n, vector<int>& weights, vector<int>& values, int W) {
    vector<int> dp(W + 1, 0);
    for(int i = 0; i < n; i++) {
        for(int j = W; j >= weights[i]; --j) {
            dp[j] = max(dp[j], values[i] + dp[W - j]);
        }
    }
    return dp[W];
}
```

Complete Knapsack You are given n items with i_{th} item having weight w_i and value v_i. Given a knapsack with max Weight capacity W. Find the max value you can have.

Note that any item can be taken **unlimited times**.

Time Complexity: $O(nW)$ **Space Complexity:** $O(W)$

```
int knapsack(int n, vector<int>& weights, vector<int>& values, int W) {
    vector<int> dp(W + 1, 0);
    for(int i = 0; i < n; i++) {
        for(int j = weights[i]; j <= W; ++j) {
            dp[j] = max(dp[j], values[i] + dp[W - j]);
        }
    }
    return dp[W];
}
```

Multiple Knapsack You are given n items with i_{th} item having weight w_i and value v_i. Given a knapsack with max Weight capacity W. Find the max value you can have.

Note that each item can be taken k_i **times**.

Time Complexity: $O(n \sum \log k_i)$ **Space Complexity:** $O(W)$

```
// Just break each item in powers of 2 and treat each division as single item and apply 01 knapsack
```

Edit Distance Given two strings s_1 and s_2, find the minimum number of operations (insert, delete, replace) required to transform s_1 into s_2.

Time Complexity: $O(mn)$ **Space Complexity:** $O(mn)$

```
int editDistance(string& s1, string& s2) {
    int m = s1.size(), n = s2.size();
    vector<vector<int>> dp(m + 1, vector<int>(n + 1));
    for (int i = 0; i <= m; ++i) dp[i][0] = i;
    for (int j = 0; j <= n; ++j) dp[0][j] = j;
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            dp[i][j] = min(dp[i - 1][j] + 1, dp[i][j - 1] + 1, dp[i - 1][j - 1] + (s1[i - 1] != s2[j - 1]));
        }
    }
    return dp[m][n];
}
```